

A Performance Study of Group Editing Algorithms

Du Li and Rui Li
Texas A&M University
Email: lidu@cs.tamu.edu

Abstract—As typical collaboration tools, group editors allow a distributed group of human users to view and edit shared documents simultaneously over a computer network. Fast local response and timely group awareness are accepted performance metrics in group editors. In this context, a family of optimistic concurrency control algorithms, called operational transformation (OT), has been well established. However, no study has been reported previously to evaluate their performance. This paper analyzes some representative OT algorithms and reveals that the performance of OT actually depends on a number of factors. The experimental results show the extent to which current OT algorithms support synchronous and asynchronous group editing.

I. INTRODUCTION

Group editors allow a distributed group of human users to edit shared documents over a computer network. A special concurrency control method called operational transformation (OT) [2], [15] has been well accepted in group editors. OT typically replicates the shared document and always allows any local editing operation to proceed immediately after it is generated. It is generally assumed that the use of OT algorithms automatically achieves high local response, high concurrency, and timely awareness. However, no underpinning performance study has been reported previously.

In this paper, we study the performance of some representative OT algorithms, including GOTO [15], SOCT2 [13], SDT [4], LBT [7], and ABT [6]. Our analyses show that the interactive performance of an OT algorithm actually depends on a number of factors such as the number of editing operations in the operation log and the ratio of their types. Our experimental results confirm the analyses and further reveal the extent to which current OT algorithms can support synchronous and asynchronous group editing tasks.

The rest of this paper elaborates this research. Section II gives an introduction to group editors and OT algorithms. Section III overviews the structures of some typical OT algorithms. Section IV presents the experimental results and analyses. Section V discusses related work. Section VI summarizes contributions and future directions of this work.

II. BACKGROUND

Following the established conventions in group editors [2], [15], we model the shared document as a string of characters (objects) and define the following two primitive operations:

- $\text{ins}(p, c)$: insert character c at position p .
- $\text{del}(p, c)$: delete character c at position p .

Let the position of the first character in a string be zero. Notation $s[i]$ returns the character at position i in string (or

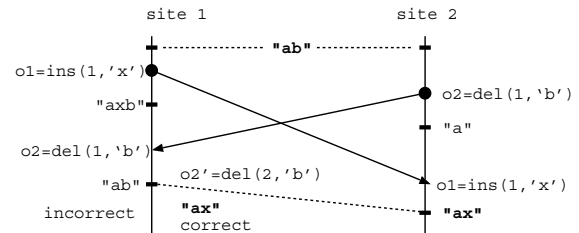


Fig. 1. OT transforms o_2 against o_1 and then executes o_2' at site 1.

state) s . For any operation o , attribute $o.t$ is its type (ins/del), $o.c$ is the effect character to be inserted or deleted, $o.p$ is its position, and $o.id$ is the id of the site that generates o . Synchronization is achieved by exchanging messages that piggyback operations. As a convention, given any two operations o_1 and o_2 , we say $o_1 \rightarrow o_2$ if o_1 happened before o_2 , and $o_1 \parallel o_2$ if o_1 is concurrent with o_2 . Apparently the position p of any operation o is always relative to some *definition state* s , denoted as $s = \text{dst}(o)$. The definition state s is the *generation state* of o if o is generated in s , or the *execution state* of o if o is to be executed in s . The fact that the execution of operation o in state s yields state s' is denoted as $s' = \text{exec}(s, o)$.

As a simple example, consider the scenario in Figure 1. Suppose two sites start from the same initial state $s_1^0 = s_2^0 = \text{"ab"}$. Site 1 performs $o_1 = \text{ins}(1, 'x')$ to insert character 'x' before 'b', yielding $s_1^1 = \text{exec}(s_1^0, o_1) = \text{"axb"}$, while site 2 simultaneously performs $o_2 = \text{del}(1, 'b')$ to delete character 'b', yielding $s_2^1 = \text{exec}(s_2^0, o_2) = \text{"a"}$. When $o_2 = \text{del}(1, 'b')$ arrives at site 1, if it is executed as it is, then the wrong character 'x' will be deleted. This is because o_2 is generated in s_2^0 without the knowledge of o_1 , but its execution state s_1^1 is different from its generation state s_2^0 , due to the execution of o_1 , which invalidates its position parameter. The intuition of OT [2] is to shift $o_2.p$ to incorporate the effect of its concurrent operation o_1 such that the result o_2' can be correctly executed in state s_1^1 . Since o_1 inserted a character 'x' before o_2 's intended character 'b', we transform o_2 against o_1 to get $o_2' = \text{del}(2, 'b')$. The execution of o_2' in s_1^1 yields the correct state $s_1^2 = \text{exec}(s_1^1, o_2') = \text{"ax"}$, which is consistent with site 2's final state after o_1 is executed: $s_2^2 = \text{exec}(s_2^1, o_1) = \text{"ax"}$.

III. AN OVERVIEW OF OT ALGORITHMS

Any two operations o_1 and o_2 are contextually equivalent, denoted as $o_1 \sqcup o_2$, iff they are defined in the same state, or $\text{dst}(o_1) = \text{dst}(o_2)$; they are contextually serialized, denoted as $o_1 \mapsto o_2$, iff o_2 is defined in the state after o_1 is executed, or s

$= \text{dst}(o_1)$ and $\text{dst}(o_2) = \text{exec}(s, o_1)$. An operation sequence sq is such that all the operations in sq are contextually serialized, or $sq[0] \mapsto sq[1] \mapsto \dots \mapsto sq[n-1]$, where $n = |sq|$. Not to be tedious, the notations of context equivalence and serialization also extend to sequences, e.g., $o \sqcup sq$, $o \mapsto sq$, $sq \mapsto o$, $sq_1 \sqcup sq_2$, and $sq_1 \mapsto sq_2$. Notation $sq_1 \bullet sq_2$ returns the concatenation of two sequences where $sq_1 \mapsto sq_2$. Extending to operations, similarly we have $sq \bullet o$ and $o \bullet sq$. Two sequences sq_1 and sq_2 are effects equivalent, denoted as $sq_1 \sim sq_2$, iff executed in the same initial state they yield the same final state. A sequence sq is causality-preserving iff the ordering of operations in sq preserves the happened-before relation, or for any $sq[i]$ and $sq[j]$, where $0 \leq i, j < |sq|$, if $sq[i] \rightarrow sq[j]$, then $i < j$ must hold.

A. The Control Procedure

Algorithm 1 The three main threads at each site

Invoke a local operation o :

- 1: $s' = \text{exec}(s, o)$;
- 2: $\langle o', H' \rangle \leftarrow \text{IntegrateLocal}(o, H)$;
- 3: broadcast o' to other sites;

Receive o from the network:

- 1: $Q' \leftarrow Q \bullet o$;

Invoke a causally-ready remote operation o :

- 1: $\langle o', H' \rangle \leftarrow \text{IntegrateRemote}(o, H)$;
 - 2: $s' = \text{exec}(s, o')$;
-

In a group editor, the control procedure at each site determines how to execute and propagate local operations that are generated by the local user, and execute remote operations that are received from the network. Most OT algorithms to our knowledge share the structure as shown in Algorithm 1.

Roughly each site i runs three threads in parallel that share three global variables: the current state s at site i , a history buffer H which logs executed operations at site i , and a remote operation queue Q which records received remote operations that are to be executed at site i . Initially s holds the shared document and both H and Q are empty.

The first thread receives operations from the local user interface. For each local operation o , it is executed immediately in current state s , which yields the next state s' . Then with algorithm $\text{IntegrateLocal}()$, the history buffer H is updated to H' and o' , a form of o , is propagated to remote sites.

The second thread receives remote operations from the network and appends them to Q in their order of arriving.

The third thread scans Q and each time select a causally-ready remote operation o for execution. Algorithm $\text{IntegrateRemote}()$ is called to transform o into o' such that o' can be correctly executed in current state s to yield the next state s' . At the same time, it adds o into the history buffer H . An operation o is causally ready at site i iff all operations that happened before o have been executed at site i .

The control procedure also maintains logical clocks and timestamps local operations such that their happened-before

and concurrent relations can be determined [2], [3], [9], [16]. To simplify synchronization, the first thread (to invoke local operations) and the third thread (to invoke remote operations) are usually executed in a mutually exclusive manner. That is, when a local operation is being invoked, no remote operation is invoked at the same time, and vice versa [19], [18].

B. Utility Transformation Functions

Inclusion Transformation (IT): The idea of $o'_1 = \text{IT}(o_1, o_2)$, where $o_1 \sqcup o_2$, is to include the effect of an executed operation o_2 into a to-be-executed operation o_1 to get o'_1 such that $o_2 \mapsto o'_1$. As an example, in Figure 1, we have $o'_2 = \text{IT}(o_2, o_1) = \text{del}(2, 'b')$.

Exclusion Transformation (ET): The idea of $o'_1 = \text{ET}(o_1, o_2)$, where $o_2 \mapsto o_1$, is to exclude the effect of o_2 from o_1 to get o'_1 such that $o'_1 \sqcup o_2$. Conceptually IT and ET are a pair of inverse transformations. For example, at site 1 in Figure 1, we have $o_2 = \text{ET}(o'_2, o_1) = \text{del}(1, 'b')$.

Swapping Two Operations (SWAP): Function $\text{SWAP}(o_1, o_2)$ is to transpose two operations $\langle o_2, o_1 \rangle$, where $o_2 \mapsto o_1$, into $\langle o'_1, o'_2 \rangle$, such that $o'_1 \mapsto o'_2$ and $[o_2, o_1] \sim [o'_1, o'_2]$. Conceptually this amounts to first processing $o'_1 = \text{ET}(o_1, o_2)$ to get $o'_1 \sqcup o_2$ and then $o'_2 = \text{IT}(o_2, o'_1)$ to get $o'_1 \mapsto o'_2$.

Algorithm 2 ITSQ(o, sq): o'

Input: o and sq are contextually equivalent, or $o \sqcup sq$;

Output: sq and o' are contextually serialized, or $sq \mapsto o'$;

- 1: $o' \leftarrow o$;
 - 2: **for** ($i=0$; $i < |sq| - 1$; $i++$) **do**
 - 3: $o' \leftarrow \text{IT}(o', sq[i])$;
 - 4: **end for**
 - 5: **return** o'
-

Algorithm 3 Convert2HC(o, sq): sq'

Input: sq is a causality-preserving sequence;

Output: $sq' \sim sq_h \bullet sq_c$, where sq_h is all operations that happened before o , and sq_c all those concurrent with o , both sq_h and sq_c being causality-preserving;

- 1: $sq_h \leftarrow \emptyset$; $sq_c \leftarrow \emptyset$;
 - 2: **for** ($j = 0$; $j < |sq|$; $j++$) **do**
 - 3: **if** $sq[j] \parallel o$ **then**
 - 4: $sq_c \leftarrow sq_c \bullet sq[j]$;
 - 5: **else**
 - 6: **for** ($k = |sq_c| - 1$; $k \geq 0$; $k--$) **do**
 - 7: $\langle sq[j], sq_c[k] \rangle \leftarrow \text{SWAP}(sq[j], sq_c[k])$;
 - 8: **end for**
 - 9: $sq_h \leftarrow sq_h \bullet sq[j]$;
 - 10: **end if**
 - 11: **end for**
 - 12: $sq' \leftarrow sq_h \bullet sq_c$;
 - 13: **return** sq' ;
-

Transforming with a Sequence (ITSQ): $\text{ITSQ}(o, sq)$ requires that o and sq be defined in the same state, or $o \sqcup sq$.

It inclusively transforms o against $sq[0], sq[1], \dots, sq[n-1]$ in turn, where $n = |sq|$, yielding $sq \mapsto o'$.

Transposing a Sequence (Convert2HC): Function $\text{Convert2HC}(o, sq)$ transposes sequence sq into $sq_h \bullet sq_c$ with regard to operation o such that all operations in sq_h happened before o and all operations in sq_c are concurrent with o . In line 1, sequences sq_h and sq_c are initialized as empty. The loop in lines 2-11 scans sq from left to right and appends any $sq[j] \parallel o$ to sq_c ; and for every $sq[j] \rightarrow o$, because $sq_c \mapsto sq[j]$, it transposes $sq[j]$ and sq_c such that $sq[j] \sqcup sq_c$ and $sq_h \mapsto sq[j]$; after that, $sq[j]$ is safely appended to sq_h .

C. The SDT Algorithm

The State Difference Transformation (SDT) algorithm [4] was proposed to address a notorious tie-breaking problem that had not been solved in any published OT work for 15 years since 1989 [2]. Under the synchronization protocol defined in Algorithm 1, here we explain how local and remote operations are integrated in SDT, respectively.

As shown in Algorithm 4, the IntegrateLocal procedure in SDT is straight-forward: it simply appends the local operation o to the history buffer without transforming o at all. Note the assumption in this paper is that any local operation o is executed immediately, meaning that the document state is not changed between the time o is generated from the user interface and the time o is invoked as in Algorithm 1. As result, all operations in the history buffer H happened before o and $H \mapsto o$ always holds.

Algorithm 4 SDT :: $\text{IntegrateLocal}(o, L): \langle o', L' \rangle$

Input: L is the history buffer and o is a local operation;

Output: $L \mapsto o'$ and $L' \sim L \bullet o'$;

- 1: $o' \leftarrow o$;
 - 2: $L' \leftarrow L \bullet o'$;
 - 3: **return** $\langle o', L' \rangle$;
-

Algorithm 5 SDT :: $\text{IntegrateRemote}(o, L): \langle o', L' \rangle$

Input: L is the history buffer and o is causally-ready;

Output: $L \mapsto o'$ and $L' \sim L \bullet o'$;

- 1: $o' \leftarrow o$;
 - 2: $L_h \bullet L_c \leftarrow \text{Convert2HC}(o, L)$;
 - 3: **for** ($i=0$; $i < |L_c|$; $i++$) **do**
 - 4: $o_x \leftarrow o'$; $o_y \leftarrow L_c[i]$;
 - 5: **if** $o_x.t = o_y.t = \text{ins}$ and $o_x.p = o_y.p$ **then**
 - 6: $sq \leftarrow L_h \bullet L_c[0, i-1]$;
 - 7: $\langle \beta_x, \beta_y \rangle \leftarrow \text{Compute}\beta(o_x, o_y, sq)$;
 - 8: derive the effects relation between o_x and o_y ;
 - 9: **end if**
 - 10: $o' \leftarrow \text{IT}(o', L_c[i])$;
 - 11: **end for**
 - 12: $L' \leftarrow L \bullet o'$;
 - 13: **return** $\langle o', L' \rangle$;
-

As shown in Algorithm 5, to integrate a remote operation o , it first calls function $\text{Convert2HC}()$ to transpose the history

buffer L into two subsequences L_h and L_c , where $L \sim L_h \bullet L_c$, L_h contains all operations that happened before o and L_c all operations concurrent with o . The input condition that o is causally-ready means that all operations that happened before o have been executed (at this site) and included in the history buffer. Hence we have $L_h \mapsto o$ and $o \sqcup L_c$. Then o is transformed with all operations in L_c to get o' , as shown in the loop of lines 3-11. After that, $L_c \mapsto o'$ and thus $L \mapsto o'$ hold. Hence we can safely append o' to the history buffer.

Now we explain what lines 4-9 mean. In early work [13], [15], $o' = \text{ITSQ}(o, L_c)$ is called naively when integrating a remote operation o . However, the notorious tie-breaking problem will arise when an insertion o is inclusively transformed with another concurrent insertion say $L_c[i]$ and their positions tie. More formally, under the condition $o_x.t = o_y.t = \text{ins}$, $o_x.p = o_y.p$, and $o_x \sqcup o_y$, it is possible that $o'_x = \text{IT}(o_x, o_y)$ cannot return the correct result [4], [7], [13]. The general pitfall in early works is that they prematurely compare the generation site ids of the two insertions as long as their positions tie.

Suppose that o_x and o_y are defined in state $s_k = \text{exec}(s_0, sq)$, where $sq = L_h \bullet L_c[0, i-1]$ and $k = |sq|$. SDT solves this tie-breaking problem by tracing back to some system state s_j that is earlier than s_k . Function $\text{Compute}\beta()$ is called to compute the positions of $o_x = o$ and $o_y = L_c[i]$ relative to s_j as if the other operations had not been executed. Conceptually this can be achieved by excluding the effects of sq (more specifically a subsequence of sq) from state s_k . However, due to some other complications, ET cannot be called directly for this purpose. As presented in [4], some special sequence sq' must be constructed from sq and then the effects of sq' is excluded instead. Finally, let β_x and β_y be the positions of o_x and o_y in state s_j , respectively, we determine the effects relation between o_x and o_y by comparing β_x and β_y and further the generation site ids of these two operations if they still tie. Once such a relation is known, IT between these two operations can always be correctly processed even if their positions tie.

D. The ABT Algorithm

The Admissibility-Based Transformation (ABT) algorithm [6] solves the correctness problem in a way drastically different from the approach of SDT. SDT maintains the history buffer H such that every newly executed operation is appended to H . By contrast, ABT maintains H as two subsequences, H_i and H_d , such that $H \sim H_i \bullet H_d$, where H_i contains all the insertions and H_d all the deletions. Every time a (local or remote) operation is integrated, it is appended to H_i or H_d depending on its type. Under the synchronization protocol defined in Algorithm 1, every local operation is propagated after excluding all the effects of H_d from it.

As shown in Algorithm 6, suppose the history buffer L is maintained as an insertions sequence L_i concatenated by a deletions sequence L_d and $L \sim L_i \bullet L_d$. When a local operation o is to be integrated, we assume that $L \mapsto o$ or $L_d \mapsto o$. The loop in lines 2-3 transposes L_d and o such that, as a result, $L_i \mapsto o'$ and $o' \mapsto L_d$. If o is a deletion, as shown in line 6, we append the original o to the original history buffer L .

If o is an insertion, as shown in line 8, we append o' to the insertion sequence L_i . In both cases, the computed o' will be propagated to remote sites.

Algorithm 6 ABT :: IntegrateLocal(o, L): $\langle o', L' \rangle$

Input: L is the history buffer and o is a local operation;

Output: $L \mapsto o'$ and $L' \sim L \bullet o'$;

```

1:  $o' \leftarrow o$ ;
2: for ( $i = |L_d| - 1; i \geq 0; i--$ ) do
3:    $\langle o', L_d[i] \rangle \leftarrow \text{SWAP}(o', L_d[i])$ ;
4: end for
5: if  $o.t = \text{del}$  then
6:    $L' \leftarrow L \bullet o$ ;
7: else //  $o.t = \text{ins}$ 
8:    $L' \leftarrow L_i \bullet o' \bullet L_d$ ;
9: end if
10: return  $\langle o', L' \rangle$ ;
```

Algorithm 7 ABT :: IntegrateRemote(o, L): $\langle o', L' \rangle$

Input: L is the history buffer and o is causally-ready;

Output: $L \mapsto o'$ and $L' \sim L \bullet o'$;

```

1:  $L_{ih} \bullet L_{ic} \leftarrow \text{Convert2HC}(o, L_i)$ ;
2:  $o'' \leftarrow \text{ITSQ}(o, L_{ic})$ ;
3:  $o' \leftarrow \text{ITSQ}(o'', L_d)$ ;
4: if  $o.t = \text{del}$  then
5:    $L' \leftarrow L \bullet o'$ ;
6: else if  $o.t = \text{ins}$  then
7:    $o_x \leftarrow o''$ ;
8:   for ( $k = 0; k < |L_d|; k++$ ) do
9:      $o_y \leftarrow o_x$ ;
10:     $o_x \leftarrow \text{IT}(o_x, L_d[k])$ ;
11:     $L_d[k] \leftarrow \text{IT}(L_d[k], o_y)$ ;
12:   end for
13:    $L' \leftarrow L_i \bullet o'' \bullet L_d$ ;
14: end if
15: return  $\langle o', L' \rangle$ ;
```

Algorithm 7 computes the form o' of a remote operation o to be executed in current state and adds o into the history buffer L in some way. Suppose L is maintained as an insertions sequence L_i concatenated by a deletions sequence L_d . Due to Algorithm 6, when o is received, it does not include the effect of any deletion. Because o is causally ready, all insertions that happened before o must be in L_i . After transposing the insertions sequence L_i into $L_{ih} \bullet L_{ic}$ in line 1, where L_{ih} groups all insertions that happened before o and L_{ic} those concurrent with o , we have $L_{ih} \mapsto o$ and $o \sqcup L_{ic}$. Hence o' , the form of o that can be correctly executed in current state $s = \text{exec}(s_0, L)$, must incorporate the effects of L_{ic} and L_d , because $L \sim L_{ih} \bullet L_{ic} \bullet L_d$. As shown in lines 2-3, o' is computed by inclusively transforming o against $L_{ic} \bullet L_d$.

Lines 4-14 decides how to add o into the history buffer L . If o is a deletion, as shown in lines 4-5, we simply append o' as computed above to L . On the other hand, if o is an insertion,

notation	measurement
t_1	execution time of IntegrateLocal()
t_2	execution time of IntegrateRemote()
t_3	communication time between two sites
t_4	buffer time at sending site
t_5	buffer time at receiving site
feedback t_6	$t_1 \leq t_6 \leq t_1 + t_2$
feedthrough t_7	$t_1 + t_2 \leq t_7 \leq t_1 + 2 * t_2 + t_3 + t_4 + t_5$

Fig. 2. Performance measurements of response and awareness.

we must somehow add o between L_i and L_d . Due to line 2, we get $L_i \mapsto o''$. However, o'' cannot be added between L_i and L_d directly, because deletions in L_d have not included the effect of o'' , or $o'' \mapsto L_d$ does not hold yet. The loop in lines 8-12 serves to include the effect of o'' into every operation in L_d such that, as a result, we have $o'' \mapsto L_d$. After that, in line 13, o'' is added between L_i and L_d .

Theoretically, ABT is a neater approach to solving the correctness problems in OT. It is proved in [7], [6] that IT problems only arise between insertions and ET problems only arise when excluding the effect of a deletion from an insertion, (roughly) if their positions tie and the ties are caused by the deletions of characters that would have separated them. ABT solves these problems fundamentally by maintaining the history buffer in a way such that those (landmark) characters have been restored when IT/ET/SWAP is processed.

IV. PERFORMANCE ANALYSES AND EXPERIMENTS

It is intuitive that the integration of any local and remote operation takes time. The research problem here is how the execution time affects the way an OT algorithm fulfills the response and awareness requirements in group editors.

A. Feedback and Feedthrough

The following two metrics are accepted in interactive systems [1]. Local responsiveness is measured by *feedback time*, i.e., the delay between an operation is generated by a user and its effect becomes visible on the user interface of that user. Timeliness of awareness is determined by *feedthrough time*, i.e., the delay between an operation is generated by a user and its effect is reflected on the interface of another user. It is established that interactive systems must provide a feedback time below 100 milliseconds (ms). Although there is no hard requirement on feedthrough time, it is understood that timely awareness is the basis for the users to resolve semantic conflicts and make sense of the group work context.

In actual group editors, such as [19], [18], the thread that integrates local operations is usually assigned a higher priority than the thread that integrates remote operations, and the local user interface is locked when a remote operation is being integrated. As shown in Figure 2, the feedback time of operation o is at least the time (t_1) it takes to integrate o locally, and the time (t_2) it takes to integrate one remote operation o' if the integration of o' has already started when o is generated. The feedthrough time of operation o is at least

its local integration time (t_1) at its generation site i and the time (t_2) to integrate o at a remote site j , plus the time (t_2) to integrate one remote operation if its integrated has started when o is generated, plus the time (t_3) to propagate o from site i to site j , plus the time (t_4) to buffer o at site i , and plus the time (t_5) to buffer o at site j . In total it is measured by $t_1 + 2 * t_2 + t_3 + t_4 + t_5$.

The communication time t_3 is ultimately determined by the speed of light. The buffer time t_4 depends on specific group work style and policies: Editing operations are often buffered before propagation to reduce bandwidth usage as well as interferences to the users even in synchronous environments [5], [19], while the collaborating sites do not continuously communicate or are often disconnected in asynchronous and mobile computing environments. In addition to these user/system-level decisions, the buffer time t_5 is also determined by the number of local and other remote operations that have to be integrated before a specific remote operation is integrated.

B. Asymptotic Time Complexities

The basic transformation functions IT/ET/SWAP take constant time to execute. The complexity of ITSQ(o, sq) is $O(|sq|)$. Assume the time to do any concatenation of operations and sequences is also constant. Both the worst-case and expected complexities of Convert2HC(o, sq) are $O(|sq|^2)$. In its worst case, all operations concurrent with o are on the left side of all operations that happened before o in sq . Hence every operation in sq_h must be transposed with sq_c . That is, $O(|sq_h| \cdot |sq_c|) = O(|sq|^2)$. In the average cases, we assume that operations are mutually independent and for every operation $sq[i]$ it is equally probable to have $sq[i] \rightarrow o$ and $sq[i] \parallel o$. When considering $sq[i]$, if uniform distribution is assumed, $\frac{i-1}{2}$ operations are in sq_h and the other $\frac{i-1}{2}$ in sq_c . Thus $sq[i]$ must be transposed with the $\frac{i-1}{2}$ operations in sq_c if $sq[i] \rightarrow o$, the probability of which is $\frac{1}{2}$. Hence the expected complexity is $\sum_{i=1}^{|sq|} (\frac{i-1}{2} \cdot \frac{1}{2} + \frac{1}{2}) \cdot O(1) = O(|sq|^2)$.

Let H_i be all insertions in the history buffer H , H_d be all deletions in H , and H_{ic} be all insertions concurrent with o . The complexity of ABT :: IntegrateLocal(o, H) is $O(|H_d|)$, and that of ABT :: IntegrateRemote(o, H) is $O(|H_i|^2 + |H_d|)$, because the complexity of Convert2HC(o, H_i) is $O(|H_i|^2)$, while lines 2-3 of Algorithm 7 take time $O(|H_{ic}| + |H_d|)$ and lines 8-12 take $O(|H_d|)$.

The execution time of SDT::IntegrateLocal(o, H) is constant. In algorithm SDT :: IntegrateRemote(o, H), the complexity of its first step Convert2HC(o, H) is $O(|H|^2)$. Let H_h be all the operations in H that happened before o and H_c be those concurrent with o . In the worst case, we have to compute the β values of o and every $H_c[i]$. Roughly function Compute $\beta()$ needs to reorder the input sequence in a way similar to function Convert2HC(). Its worst-case and average-case complexities are both quadratic in the size of its input sequence. Hence the complexity of the loop of lines 3-11 of Algorithm 5 is $\sum_{i=1}^{|H_c|} (|H_h| + i)^2 \cdot O(1) = O(|H|^3)$, which dominates the quadratic time of Convert2HC(o, H). Therefore the worst-case time complexity of Algorithm 5 is $O(|H|^3)$.

In average cases, we assume that all operations are mutually independent and all distributions are uniform. Specifically, the operation types are uniformly distributed, i.e., $\Pr[o.t=ins] = \Pr[o.t=del] = \frac{1}{2}$, and so are the operation positions, i.e., $\Pr[o.p=i] = \frac{1}{D}$, where $i = 0, 1, 2, \dots, D$ and D is the length of the document. Then the probability of running into the insertion tie is $\Pr[o_1.t=o_2.t=ins] \cdot \Pr[o_1.p=o_2.p] = \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{D} = \frac{1}{4 \cdot D}$. Hence the expected complexity of SDT::IntegrateRemote() is $O(|H|^2 + \frac{1}{4 \cdot D} \cdot |H|^3)$, which is reduced to $O(|H|^2)$ when D is large enough and the insertion ties become rare.

C. Performance Experiments

Experiments are necessary to understand what the asymptotic complexities mean when interactive constraints are present in the system. By the above analyses, the number of cooperating sites does not contribute directly to feedback and feedthrough but size and composition of the history buffer at each site. That said, scalability of OT algorithms is not an issue in this paper assuming that they are complemented by a scalable communication infrastructure. Therefore, in the following experiments, we only consider two sites and use the total execution time of IntegrateLocal() and IntegrateRemote() as the measurement of feedback and feedthrough. We will show that, under certain conditions, this part ($t_1 + t_2$) can become significant enough to dominate $t_3 + t_4 + t_5$ in Figure 2.

The algorithms were implemented in C++, compiled by GNU g++ v3.3.3, and executed on a computer running RedHat Linux kernel 3.3.3-7 with an Intel Pentium-4 3.4 GHz CPU and 1 GB RAM. There are two sites in all the experiments: site A generates a sequence of M operations and concurrently site B generates a sequence of N operations. In the first step of each experiment, the N -sequence by site B is integrated into the M -sequence at site A one by one. This way we simulate a history buffer of $M + N$ operations at site A. In the second step, a new operation o is generated at site B and integrated at site A with a history of $M + N$ operations, in which the M operations that are concurrent with o precede the N operations that happened before o . This way function Convert2HC(o, H) always executes in its worst case.

1) *Performance of SDT*: According to the above analyses, the execution time of SDT::IntegrateLocal() is a small constant that is negligible. The feedback and feedthrough of SDT are measured by the execution time of IntegrateRemote(), which depends on the size of H and the rate of insertion ties.

We did three experiments as shown in Figure 3. The left chart shows the worst-case execution time of IntegrateRemote(o, H), i.e., when the remote operation o to be integrated is an insertion, and it ties (Algorithm 5 line 5) with each of the M concurrent insertions in H . This experiment shows that, in the worst case, SDT can guarantee a response time of 100ms only when M and N are both very small, e.g., about 50 operations on a 3.4GHz PC. The cooperating sites must frequently communicate and synchronize their operations in order to maintain a history buffer of this size.

In the other two experiments, we study how the performance varies by decreasing the probability of insertion ties. In both

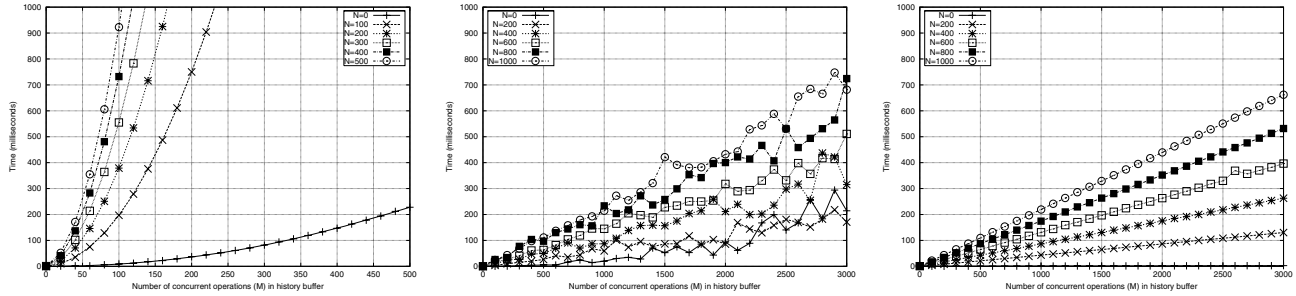


Fig. 3. Execution time of $\text{SDT::IntegrateRemote}(o, H)$ with (L) worst-case, (M) high, and (R) low rates of insertion ties, $|H| = M + N$.

experiments, the remote operation to be integrated and all operations in the history buffer are generated with a probability 90% of being an insertion. The positions of all operations are uniformly distributed over the current document. The rate of insertion ties is controlled by varying the size of the document. For each history buffer composition, the experiments are run 30 times to average the execution times. In the middle chart, the document starts from 100 characters, while in the right chart, it starts from 1,000,000 characters. Hence in the former the rate of insertion ties is high while in the latter the rate is low. These two charts clearly show that the performance of SDT fluctuates as the rate of insertion ties increases. They also suggest, however, that the fluctuation is bound. Hence in non-worst cases, more flexibility in synchronization is allowed. For example, to provide an interactive response time of 100ms, both M and N can be several hundreds.

2) *Performance of ABT*: By the above theoretical analyses, the execution time of $\text{IntegrateLocal}(o, H)$ is dominated by that of $\text{IntegrateRemote}(o, H)$, which is in turn dominated by the performance of $\text{Convert2HC}(o, H_i)$. Since ABT does not break insertion ties in special cases as does SDT, its performance is mostly determined by the percentage of insertions in the history buffer rather than the rate of insertion ties.

Figure 4 shows two experiments with different percentages of insertions in H . As explained above, H is maintained such that all insertions H_i are ordered before all deletions H_d . Additionally, H_{ic} , all operations concurrent with o , are ordered before H_{ih} , all operations that happened before o . Hence $\text{Convert2HC}(o, H_i)$ always executes in its worst case.

Both experiments measure the total time to execute $\text{IntegrateLocal}(o, H)$ and $\text{IntegrateRemote}(o, H)$ with the same configuration of H . By the analyses in Section IV-A, this measurement reflects the feedback and feedthrough of ABT. All operations in the experiments are generated such that their positions are uniformly distributed over the current document. In the left chart, every operation is an insertion, which is the worst case of ABT. To guarantee an execution time of 100ms, roughly the values of M and N can be a few hundreds. And in the right chart, every operation is equally likely to be an insertion or a deletion, which is comparatively an average case of ABT. The execution time falls within 100ms for all $M \leq 3000$ and $N \leq 1000$, which is not achieved in SDT even when its rate of insertion ties is low.

The experiments confirm the theoretical analysis that the performance of ABT depends on the percentages of concurrent operations and insertions in the history buffer. In general, the worst-case performance of ABT improves over the average-case SDT performance by some factor. This is due to the difference between the use of $\text{Convert2HC}(o, H)$ in SDT and the use of $\text{Convert2HC}(o, H_i)$ in ABT, where H_i is but a subsequence of H . Similarly, as the percentage of insertions decreases, the execution time in the right chart improves by some other factor over that in the left chart.

3) *Performance of integrating two sequences*: All the above experiments clearly indicate that the feedback and feedthrough time is nonnegligible in both SDT and ABT. However, it is not clear yet how much time it takes to integrate one operation sequence into another sequence. This is important for two reasons: (1) in group editors, synchronous or asynchronous, operations are often propagated in batches rather than on a per-operation basis; and (2) as analyzed in Section IV-A, the number of operations in the sending buffer or receiving queue does not affect feedback but it does affect feedthrough (awareness) of remote operations.

As shown in Figure 5, we did two experiments to measure the time to integrate a sequence of N operations with a concurrent sequence of M operations. Both experiments integrate the N operations one by one. The left chart uses SDT with the low rate of insertion ties as in the right chart of Figure 3. More concretely, it takes over 50 seconds to integrate a sequence of 400 operations with a sequence of 3000 operations, or over 100 seconds to integrate two sequences each 1000 operations. In the right chart, ABT is used under the same insertion percentage as in the right chart of Figure 4. The execution time of ABT is faster than that of SDT roughly by a factor of six. Nevertheless, the time is still nonnegligible. For example, it takes about 10 seconds to integrate a sequence of 3000 operations with a sequence of 400 operations, or about 20 seconds to integrate two sequences each with 1000 operations.

D. Conclusions

We draw the the following three basic conclusions from the above performance analyses and experiments:

First, back to Figure 2, our experiments show that, given certain size of the history buffer and composition of its operations, the execution time of integrating a local and a

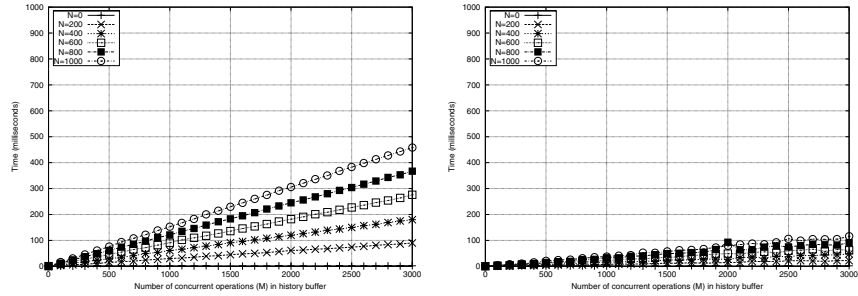


Fig. 4. Execution time of $ABT::IntegrateLocal(o, H)+IntegrateRemote(o, H)$ with (L) 100% and (R) 50% insertions, $|H| = M + N$.

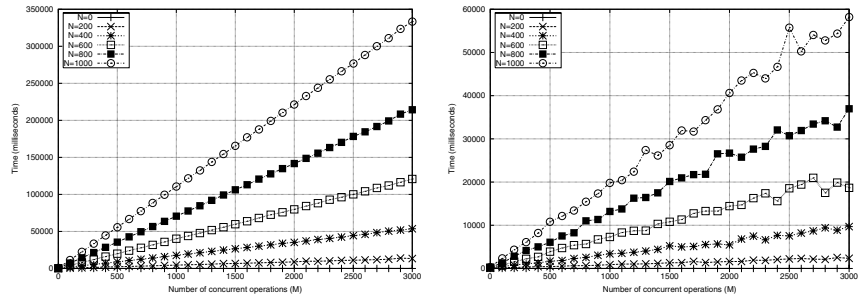


Fig. 5. Execution time of integrating N operations with M concurrent operations: (L) SDT with low insertion ties, and (R) ABT with 50% insertions.

remote operation ($t_1 + t_2$) dominates typical network latencies (t_3) that are at the magnitude of 50-200ms. Even when buffer time ($t_4 + t_5$) is considered, $t_1 + t_2$ is still a nonnegligible contributor to feedback and feedthrough of a group editor.

Second, both SDT and ABT are able to achieve interactive response and awareness in realtime group editing. The performance of SDT is less stable than that of ABT. However, this difference only changes the synchronization policies within a certain range. For example, with SDT, the users are expected to flush their sending buffers at every tens of operations. With ABT, this limit is extended to a few hundred. Only within this range can these algorithms guarantee a local response time of 100ms and an awareness time of a few seconds.

Third, these two algorithms are not designed for asynchronous and mobile collaborative computing. In such environments, the users are often disconnected and long sequences of operations could accumulate nondeterministically. Due to the experiments shown in Figure 5, we know that it could take a few minutes in SDT or tens of seconds in ABT to integrate two concurrent sequences with a couple thousand of operations. Some extensions to these algorithms would be required if they were to support non-realtime collaboration.

V. RELATED WORK

Now we explain why SDT and ABT are representative OT algorithms and this performance study is relevant.

Previous OT algorithms are required to satisfy three conditions: convergence, causality and intention preservation [16], [15]. However, the intention preservation condition is not well-formalized. As a consequence, the correctness of previous OT algorithms with regard to intention preservation cannot

be formally proved, including dOPT [2], AdOPTed [10], SOCT2/3/4 [13], [17], GOT [16], GOTO [12], [15], [14], and TIBOT [8]. In fact, in [4], we gave counterexamples to all of them showing they failed in some tie-breaking scenarios.

However, SOCT2 and GOTO are still interesting: (1) SOCT2 as published [13] does not provide critical algorithm details for solving the tie-breaking problem, and so there is a chance that the completed algorithm is correct; and (2) GOTO has been continuously refined since [15], and so there is a chance that the tie-breaking problem has been fixed but the solution has not been published.

The correctness problem was not solved until our recent work in SDT [4], LBT [7], and ABT [6]. Our works came with newly established correctness conditions and formal proofs. In this paper, we only study the performance of SDT and ABT for two reasons: (1) ABT and LBT differ mainly in proof techniques and ABT is more efficient than LBT; and (2) SDT has similar structures as previous algorithms such as SOCT2 and GOTO, which is explained as follows.

Early algorithms such as dOPT and AdOPTed have a control procedure that is different from Algorithm 1. They add both local and remote operations to the queue and transform them in the same way against the history buffer (although local operations are given higher priority). Hence their local response time is conceptually worse than more recent OT algorithms such as SOCT2 and GOTO that take the control procedure shown in Algorithm 1.

The Convert2HC() function was first proposed in SOCT2 [13] and GOTO [15] to reorder the history buffer such that some problems in the original dOPT [2] are fixed. Hence

the first step of GOTO and SOCT2 are the same as that of SDT. These three algorithms differ primarily in the ways they address the insertion-tie problem. Because the cubic execution time of SDT for solving the tie-breaking problem rarely occurs, our experiments show that the performance of SDT is rather stable due to the quadratic time of Convert2HC(). Hence we believe that the performance of SOCT2 and GOTO should be similar to that of SDT, if they also have solved the insertion-tie problem.

In fact, GOT and its successor GOTO are the only published OT algorithms that support stringwise operations. All the other OT algorithms to our knowledge only consider characterwise insertions and deletions. Hence we only consider characterwise operations in this performance study so that the study is representative. As shown in GOT [16], the handling of stringwise and characterwise operations only differ in the basic IT/ET functions. The handling of strings in IT/ET only increases their time complexity by a negligible constant factor. Therefore we believe that the above performance conclusions about OT algorithms generally hold for all algorithms that support characterwise and stringwise operations.

VI. CONTRIBUTIONS AND FUTURE WORK

This work makes the following two contributions. First, for the first time, it studies the performance of a family of OT algorithms that are widely used in group editors. The boundary between synchronous and asynchronous collaboration was previously not well understood in these systems. There were often proposals to support asynchronous and mobile computing without underpinning performance evaluation, e.g. [11], [13]. Experimental results in this paper give a more concrete idea as what complexities of OT algorithms mean in the face of interactive constraints, and roughly how frequently the users must synchronize their operations in order to satisfy these constraints. This work confirms that existing OT algorithms are sufficient for synchronous collaboration but not generally suitable for supporting asynchronous work.

Secondly, this paper presents an overview of existing OT algorithms. The overview is the basis of the performance evaluation and is interesting on its own right. Although SDT and ABT were presented previously elsewhere, their formalization in this paper is different, which provides a new, unified view of OT algorithms for the purposes of performance evaluation and comparison. The first survey of OT algorithms appeared in [15]. Since then a large body of work has been developed on this topic. An update on the state of the art is necessary for researchers and practitioners. This work as a survey suggests the worthwhile directions for improvement in OT algorithms and group editors in general.

In future work, we plan to extend the ABT algorithm with stringwise operations and particularly optimize it to support the integration of sequences. These extensions will make the algorithm more suitable for asynchronous and mobile collaborative computing environments. We also plan to extend the performance evaluation part by additionally considering the impacts of networking and buffering.

REFERENCES

- [1] A. Dix, J. Finlay, G. Abowd, and B. Russell. *Human-Computer Interaction*. Prentice Hall, 1998.
- [2] Clarence A. Ellis and S. J. Gibbs. Concurrency control in groupware systems. In *Proceedings of the ACM SIGMOD'89 Conference on Management of Data*, pages 399–407, Portland Oregon, 1989.
- [3] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, July 1978.
- [4] Du Li and Rui Li. Preserving operation effects relation in group editors. In *Proceedings of the ACM CSCW'04 Conference on Computer-Supported Cooperative Work*, pages 457–466, November 2004.
- [5] Du Li, Chengzheng Sun, Limin Zhou, and Richard R. Muntz. Operation propagation in real-time group editors. *IEEE Multimedia Special Issue on Multimedia Computer Supported Cooperative Work*, 7(4):55–61, 2000.
- [6] Rui Li and Du Li. Commutativity-based concurrency control in groupware. In *Proceedings of the First IEEE Conference on Collaborative Computing: Networking, Applications and Worksharing (Collaborate-Com'05)*, San Jose, CA, December 2005.
- [7] Rui Li and Du Li. A new operational transformation framework for real-time group editors. *IEEE Transactions on Parallel and Distributed Systems*, March 2006. Accepted.
- [8] Rui Li, Du Li, and Chengzheng Sun. A time interval based consistency control algorithm for interactive groupware applications. In *IEEE International Conference on Parallel and Distributed Systems (ICPADS)*, pages 429–436, July 2004.
- [9] Michel Raynal and Mukesh Singhal. Logical time: Capturing causality in distributed systems. *IEEE Computer*, pages 49–56, February 1996.
- [10] Matthias Ressel, Doris Nitsche-Ruhland, and Rul Gunzenhäuser. An integrating, transformation-oriented approach to concurrency control and undo in group editors. In *Proceedings of the ACM CSCW'96 Conference on Computer-Supported Cooperative Work*, pages 288–297, November 1996.
- [11] Haifeng Shen and Chengzheng Sun. Flexible merging for asynchronous collaborative systems. In *Proceedings of International Conference on Cooperative Information Systems (CoopIS'02)*, pages 304–321, Irvine, California, October 2002.
- [12] Haifeng Shen and Chengzheng Sun. Flexible notification for collaborative systems. In *Proceedings of the ACM CSCW'02 Conference on Computer-Supported Cooperative Work*, pages 77–86, November 2002.
- [13] Maher Suleiman, Michèle Cart, and Jean Ferrié. Concurrent operations in a distributed and mobile collaborative environment. In *IEEE ICDE'98 International Conference on Data Engineering*, pages 36–45, February 1998.
- [14] Chengzheng Sun. Undo as concurrent inverse in group editors. *ACM Transactions on Computer-Human Interaction*, 9(4):309–361, December 2002.
- [15] Chengzheng Sun and Clarence Ellis. Operational transformation in real-time group editors: issues, algorithms, and achievements. In *Proceedings of the ACM CSCW'98 Conference on Computer-Supported Cooperative Work*, pages 59–68, December 1998.
- [16] Chengzheng Sun, Xiaohua Jia, Yanchun Zhang, Yun Yang, and David Chen. Achieving convergence, causality-preservation, and intention-preservation in real-time cooperative editing systems. *ACM Transactions on Computer-Human Interaction*, 5(1):63–108, March 1998.
- [17] Nicholas Vidot, Michelle Cart, Jean Ferrie, and Maher Suleiman. Copies convergence in a distributed realtime collaborative environment. In *Proceedings of ACM CSCW'00 Conference on Computer-Supported Cooperative Work*, pages 171–180, December 2000.
- [18] Steven Xia, David Sun, Chengzheng Sun, David Chen, and Haifeng Shen. Leveraging single-user applications for multi-user collaboration: the CoWord approach. In *Proceedings of ACM CSCW'04 Conference on Computer Supported Cooperative Work*, pages 437–446, November 2004.
- [19] Yun Yang, Chengzheng Sun, Yanchun Zhang, and Xiaohua Jia. Real-time cooperative editing on the Internet. *IEEE Internet Computing*, 4(3):18–25, 2000.